



Instructor: Date:

Learner: Regional Branch:

Program: Center:

CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points

**Let's Start!****Programming Paradigms and Their Characteristics**

1011010001000111010
010010 Programming 1011010
101101 Paradigm 1101110
0101101101001011011

Learning Outcome:

Describe common activities at work to a visitor using short sentences in familiar spoken and written situations.

Explain important features related to object programming in different contexts and with different types of object programming.

Can Do Statements:

- I can explain the notion of software paradigm.
- I can explain different types of paradigms related to software programming.
- I can give detailed information related to program paradigms.

Ask your partner:

- What do you think are the main features of programming languages?
- How can you classify programming languages? Make a list.





Let's Explore!

A Programming Paradigm Training Session

object-oriented

symbolic

logic



declarative

Instructions: Read the text below.

Good morning, ladies and gentlemen! Today, we will talk about software paradigms, also known as programming paradigms, and their different types. If you don't understand what I'm saying, please stop me, and I'll clarify any doubts. Let's begin! As you know, there is too much information about programming paradigms, but first, we are going to start with the basics. We use programming paradigms as a method to classify programming languages which are categorized by the style of the computer programming. In other words, if a language doesn't have particular traits, you can't determine what programming paradigm it belongs to. There are too many paradigms that focus on the language's execution model. However, there are others that are more concerned about how the code is organized by groups.

Among the most common programming paradigms are imperative paradigm, which allows side effects; declarative paradigm, which doesn't state the order of the executed operations; functional paradigm, which excludes the side effects. Additionally, we have object-oriented paradigm, which organizes groups of codes; different from procedural paradigm, which organizes codes into functions. We also have logic paradigm, which needs a certain execution model. Finally, we have symbolic paradigm, which has a particular style of grammar and syntax. I know there is too much to learn; however, I'm going to give you a few examples later.

Languages that are part of imperative paradigm have two important characteristics: the first one is that it allows side effects and also guides the order of the operations with constructs that command that order; and the second one consists of controlling the flow in an explicit sequence of commands. Meanwhile, declarative paradigm states the result of the executed operations, but it doesn't state the order of the process itself. In other words, if you don't want to know the conditions of the number of operations that can be used, you will use declarative paradigm. In object-oriented paradigm, you build the codes as objects, which you can change for the same code in the same object.

Well, I think that's all for today. Remember, there are too many programming paradigms, review them all the time. If you don't research more about these concepts, you won't learn how to classify programming languages easily!.





Let's Practice!

There is too much to know about programming paradigms.

Instructions: Classify the sentences by writing the number of the sentence in the correct category

1. Programming languages need to have many numbers to become a paradigm.
2. A programming paradigm classifies programming languages into groups.
3. A language needs to have particular traits in order to determine what programming paradigm it belongs to.
4. A software paradigm doesn't classify programming languages.
5. A software paradigm is also known as programming paradigm.
6. A software paradigm is also known as software engineering.

True Facts About Software Paradigm	False Facts About Software Paradigm



Let's Practice!

The Conditions of Programming Paradigms

Instructions: Use the words from the word bank to complete each sentence.

1. If the declarative paradigm doesn't allow side effects, _____.
 - a. the imperative paradigm will execute it.
 - b. the object-oriented paradigm will do it.
 - c. the functional paradigm will perform it.
2. _____ then the procedural paradigm won't organize the codes but will organize their functions.
 - a. If the functional paradigm organizes groups of codes,
 - b. If the object- oriented paradigm organizes groups of codes,
 - c. If the symbolic paradigm organizes groups of codes,
3. If the symbolic paradigm doesn't have a particular style of grammar and syntax, _____.
 - a. it doesn't work
 - b. it will work
 - c. it won't work





Extension Activity

Let's learn more about programming paradigms.

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Look for more information about programming paradigms.
- List the main characteristics of programming paradigms.
- Give examples of computing programming languages using the main characteristics of programming paradigms.
- Share with your partners and your teacher.

Self-Assessment

Yes

No

- I look for more information about programming paradigms.
- I list the main characteristics of programming paradigms.
- I give examples of computing programming languages using the main characteristics of programming paradigms.
- I share with my partners and teacher.

☐ ☐
☐ ☐
☐ ☐
☐ ☐


Target Words

- **clarify** (verb): to explain
e.g. If you don't understand what I'm saying, please stop me, and I'll clarify any doubts.
- **trait** (noun): characteristic or certain quality
e.g. In other words, if a language doesn't have particular traits, you can't determine what programming paradigm it belongs to.
- **paradigm** (noun): a group of something which contains particular concepts or elements
e.g. Today, we will talk about programming paradigms and their different types.
- **execute** (verb): to perform, to carry out
e.g. Meanwhile, declarative paradigm states the result of the executed operations.
- **syntax** (noun): the study of language rules in the formation of sentences in a determined language
e.g. We also have logic paradigm which has a particular style of grammar and syntax.



Learning Points

The First Conditional in Negative

We use the **first conditional** to talk about a possible condition and its outcome in the future.

The **first conditional** has two clauses, the first clause (condition) with '*if*' in *present simple* and the second clause (result) with the tense in *future simple*. In this case, we are going to study the **negative form**. Let's see the structures:

If + Subject	Present Simple Negative,	Subject+ will	Complement
If you	don't understand the concept of program paradigm,	you'll	make a mistake in the future.
If + Subject	Present Simple,	Subject + will negative	Complement
If they	know the classifications of program paradigms,	they won't	need to apply all of them unnecessarily.

Note: The order of the two clauses is flexible and it's not important, but if the if-clause comes first, it must be followed by a comma.

Examples:

If you **don't understand** how to use programming languages, **I'll explain** it to you.

I'll explain it to you if you **don't understand** how to use programming languages.

Remember that you can also use the negative form in **both clauses**. Like this:

If you **don't study** the different programming paradigms, **you won't** learn how classify programming languages.

Examples

- If you don't understand what I'm saying, please stop me, and I'll clarify any doubts.
- In other words, if a language doesn't have particular traits, you can't determine what programming paradigm it belongs to.
- If you don't want to know the conditions of the number of operations that can be used, you will use declarative paradigm.
- If you don't research more about these concepts, you won't learn how to classify programming languages easily!



Learning Points

Quantifiers

A **quantifier** is a word or phrase that indicates the amount or number we want to give to someone or something.

Quantifiers can be used with both, count nouns and non-count nouns. Today, we are going to study **too much** and **too many**.

Too much is used with non-count nouns. Let's see the next example:

*There is **too much** information about programming paradigms.*

As you can see, you always use **there is** with the quantifier **too much**.

Too many is used with count nouns. Let's see the next example:

*There are **too many** paradigms involved in the execution model of a language.*

As you can see, you always use **there are** with the quantifier **too many**.

Examples

- *As you know, there is too much information about programming paradigms, but first, we are going to start with the basics.*
- *There are too many paradigms that focus on the language's execution model.*
- *I know there is too much to learn; however, I'm going to give you a few examples later.*
- *Remember, there are too many programming paradigms, review them all the time.*